

Compute 3

Hannah Srzich

February 2026

Question 1.

Inverse Problem for the Diffusion Coefficient and the `tinyDA` package

If you have not finished sampling the posterior distribution for the diffusion coefficient D and uncertain final time T , now is your chance. Plot a histogram of the marginal posterior over each of D and T .

If you are a Pythonista, you might like to try out the package `tinyDA` that implements delayed acceptance MCMC, adaptive proposals, multi-level MCMC, and more! Usefully, `tinyDA` interfaces to the `ArviZ` package for visualising the output of the MCMC, so you can produce standard joint scatter plots and marginal histograms, without doing the work yourself.

Below is output from the Metropolis-within-Gibbs implementation as in compute 2, part (c)...

Part 2(c): Metropolis-within-Gibbs for joint (D , $T_{\max}$)

Configuration:

```
True D:      1.0
True Tmax:   2.0
Chain length: 20000
Burn-in:     2000
wD (D RW):   0.2
wT (T RW):   0.1
Prior support: D>0, Tmax in [1.9, 2.1]
```

Posterior summaries (post burn-in):

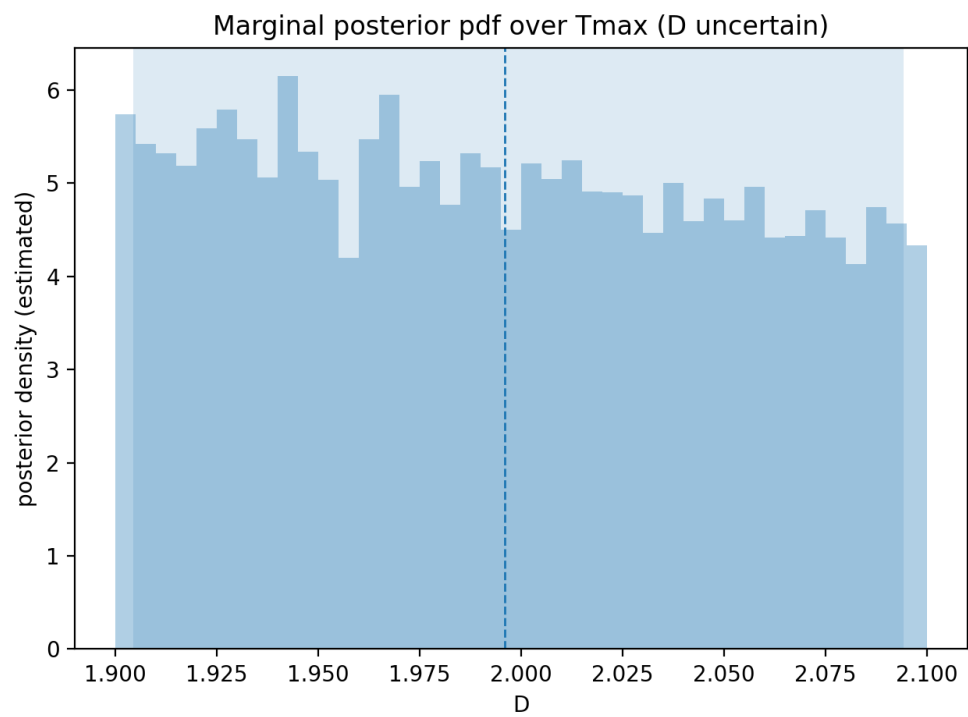
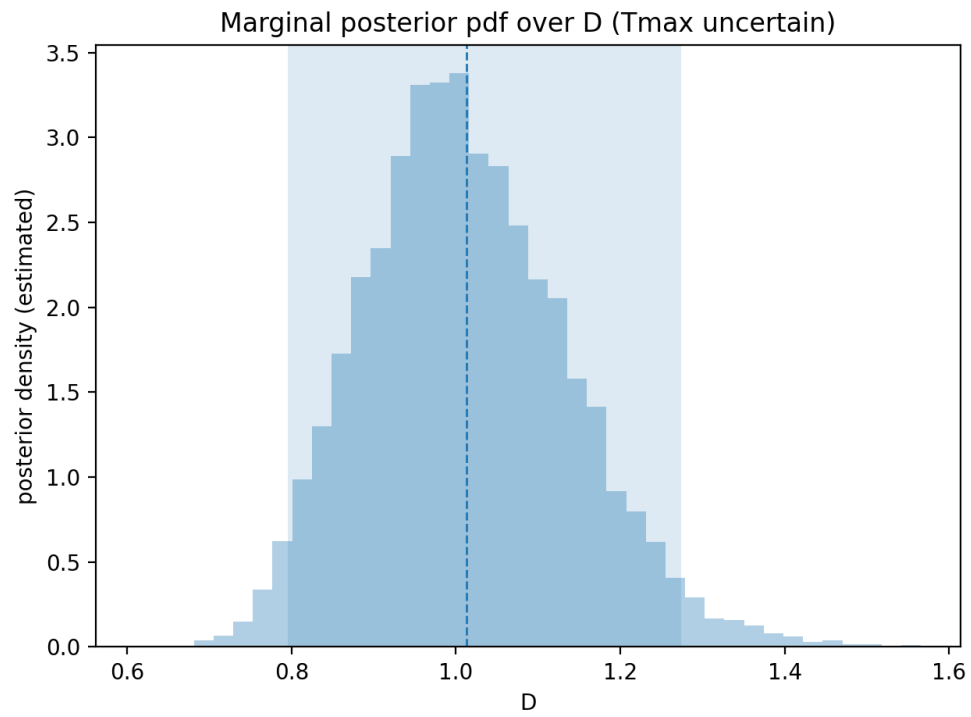
```
D mean:  1.01274   | 95% CI: [0.795208, 1.27425]
T mean:  1.99595   | 95% CI: [1.90436, 2.09427]
```

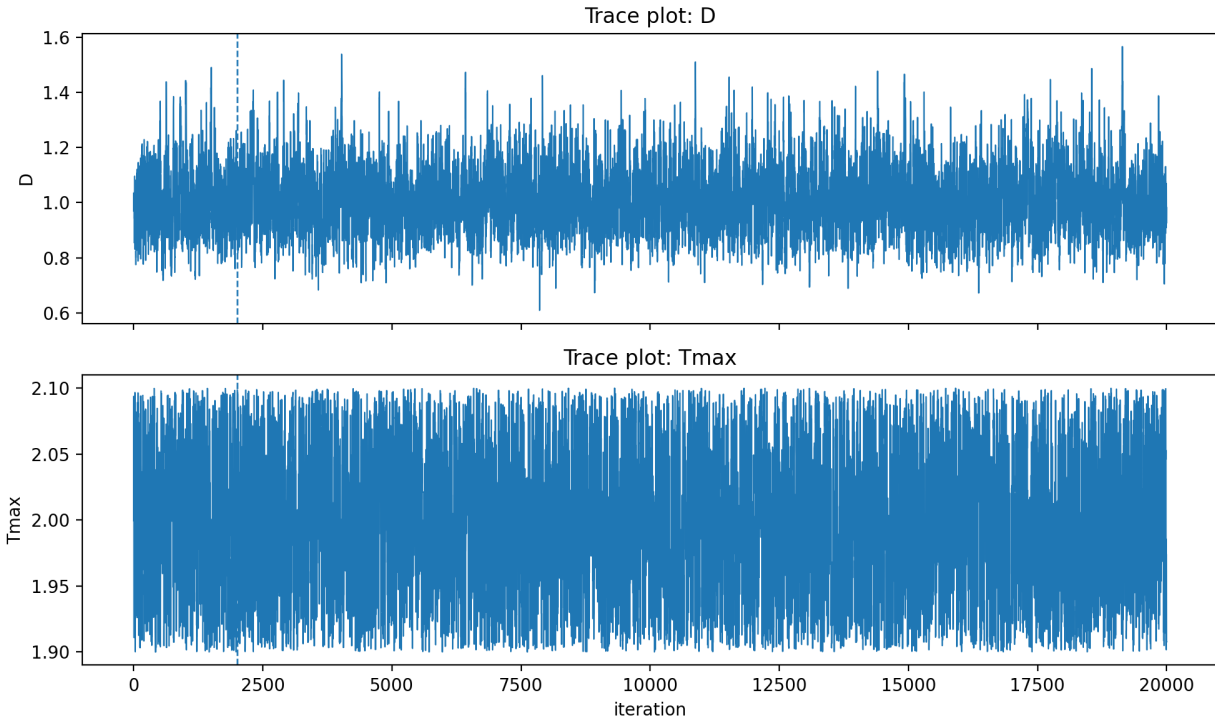
Acceptance rates:

```
D update (all iters):    0.6822
Tmax update (all iters): 0.6897
D update (post burn-in): 0.6849
Tmax update (post burn-in):0.6928
```

IACF estimates (`puwr.tauint`, post burn-in):

```
tau_int(D)      = 8.50708
tau_int(Tmax)   = 7.30982
```





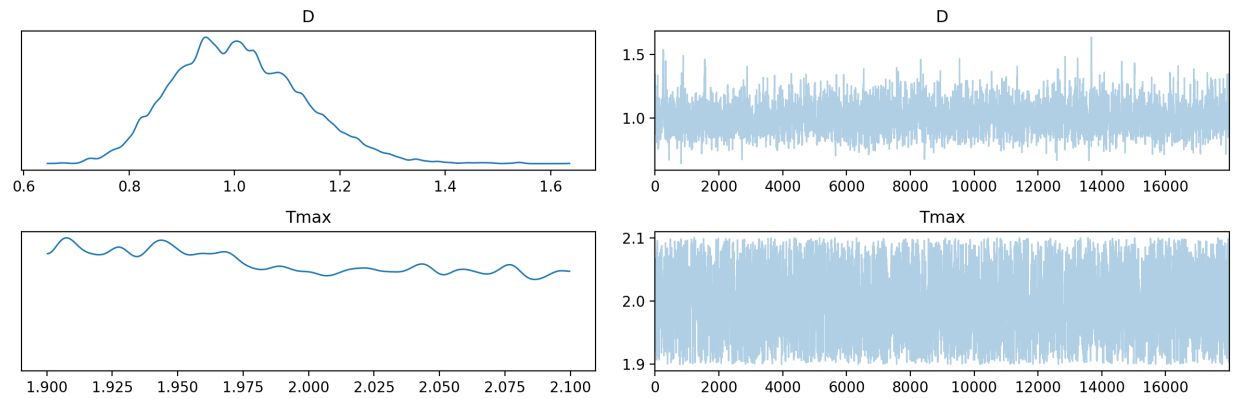
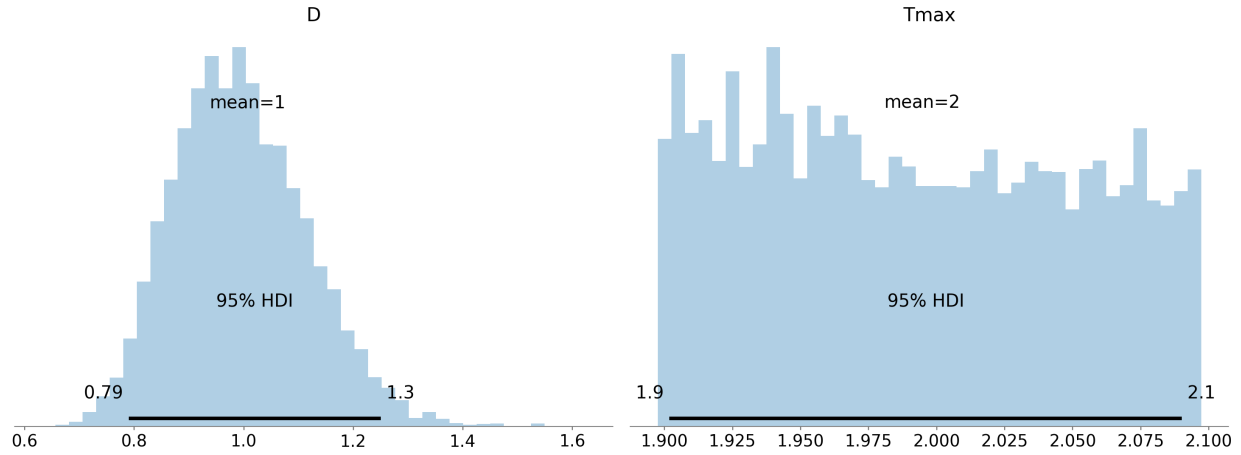
We then implement a Random-Walk Metropolis–Hastings sampler (with a Gaussian proposal) using `tinyDA` and `ArviZ`...

Running `tinyDA` RWMH on (D, Tmax)

```
true:  D=1.0, Tmax=2.0
prior: D>0, Tmax in [1.9, 2.1]
N=20000, burnin=2000, chains=1
RW std: wD=0.2, wT=0.1
Sampling chain 1/1
Running chain, alpha = 0.35
```

ArviZ summary (post burn-in):

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
D	1.007	0.122	0.795	1.236	0.003	0.002	1962.0	2054.0	NaN
Tmax	1.995	0.058	1.902	2.088	0.001	0.001	2199.0	2380.0	NaN



Question 2.

Finite Rank Property

Now consider the inverse problem for the diffusion coefficient that is assumed to be a (positive) function $D(x)$ of x (so no longer a constant), but T is fixed. By using a suitable discretization of $D(x)$, calculate the Jacobian of the map from $D(x)$ to noise-free measurements $\{u(x_i, T)\}$ (perhaps using a finite difference approximation), at the nominal true value of D , and then evaluate the singular vector decomposition to determine the effective rank at the SNR considered. Does the rank depend on your discretization of $D(x)$ and $u(x, t)$, for sufficiently fine discretizations?

Below we repeat the spatial discretization of the forward problem for increasing values, and then compute the Jacobian of the map from $D(x)$ to noise-free measurements $\{u(x_i, T)\}$ at the true value of $D = 0.5$ using the secant method. We then compute its singular values via SVD to calculate the effective rank.

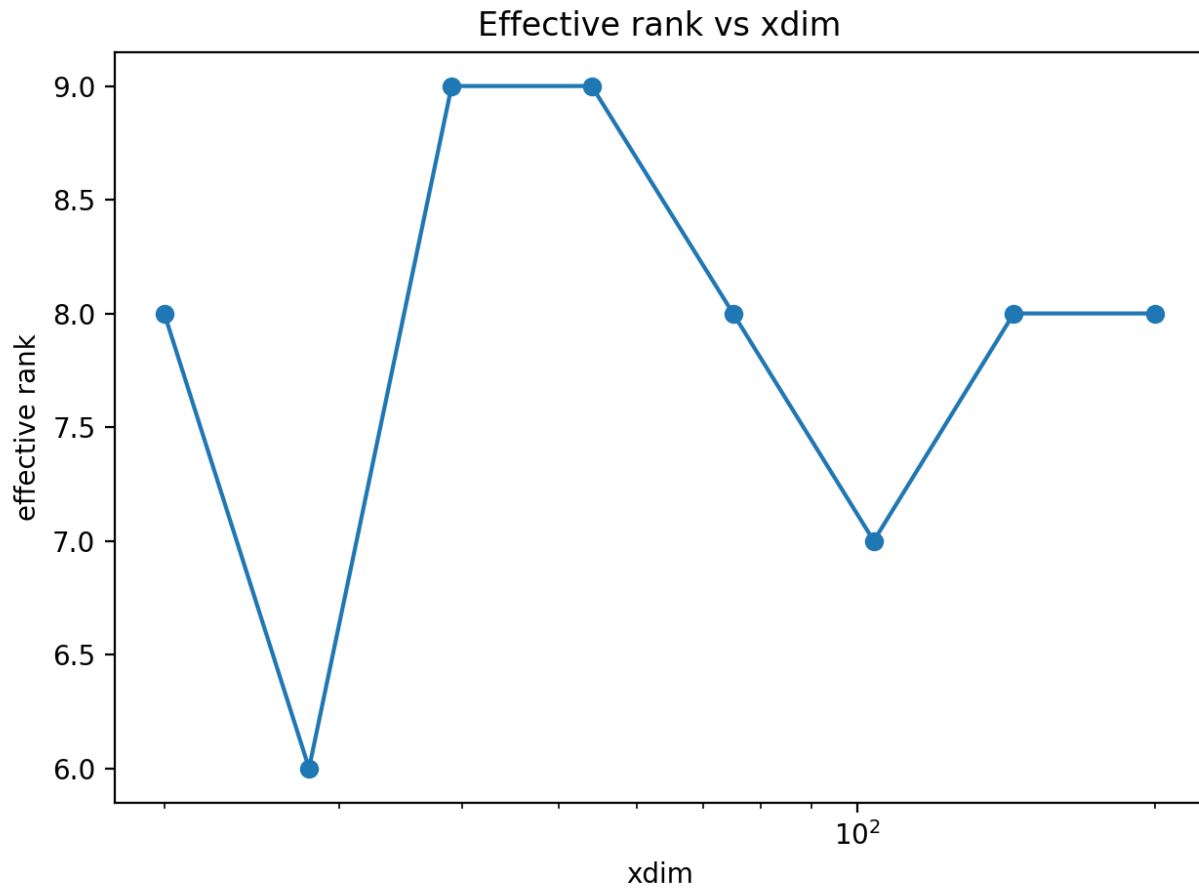
Compute 3 (Part 2):

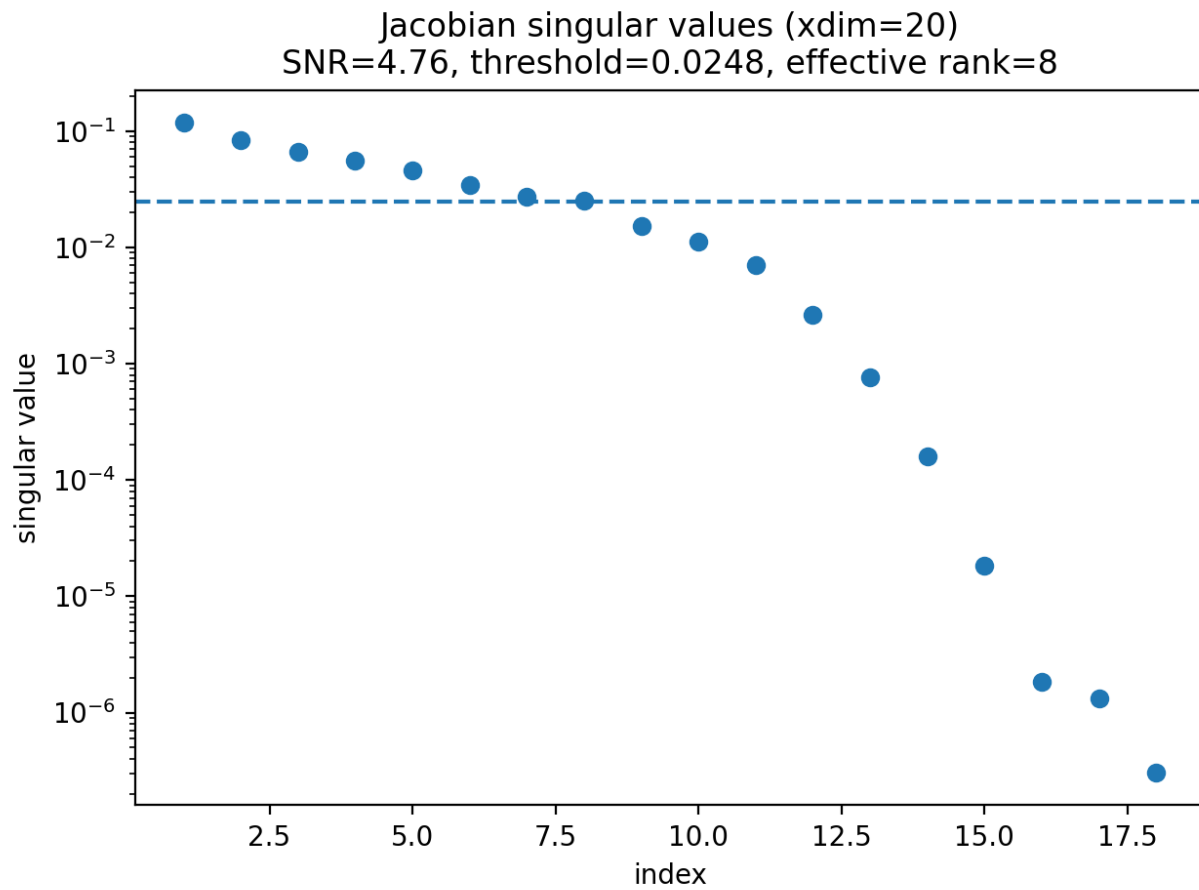
$D_{\text{true}} = 0.5$
 $L, T_{\max} = 10.0, 2.0$

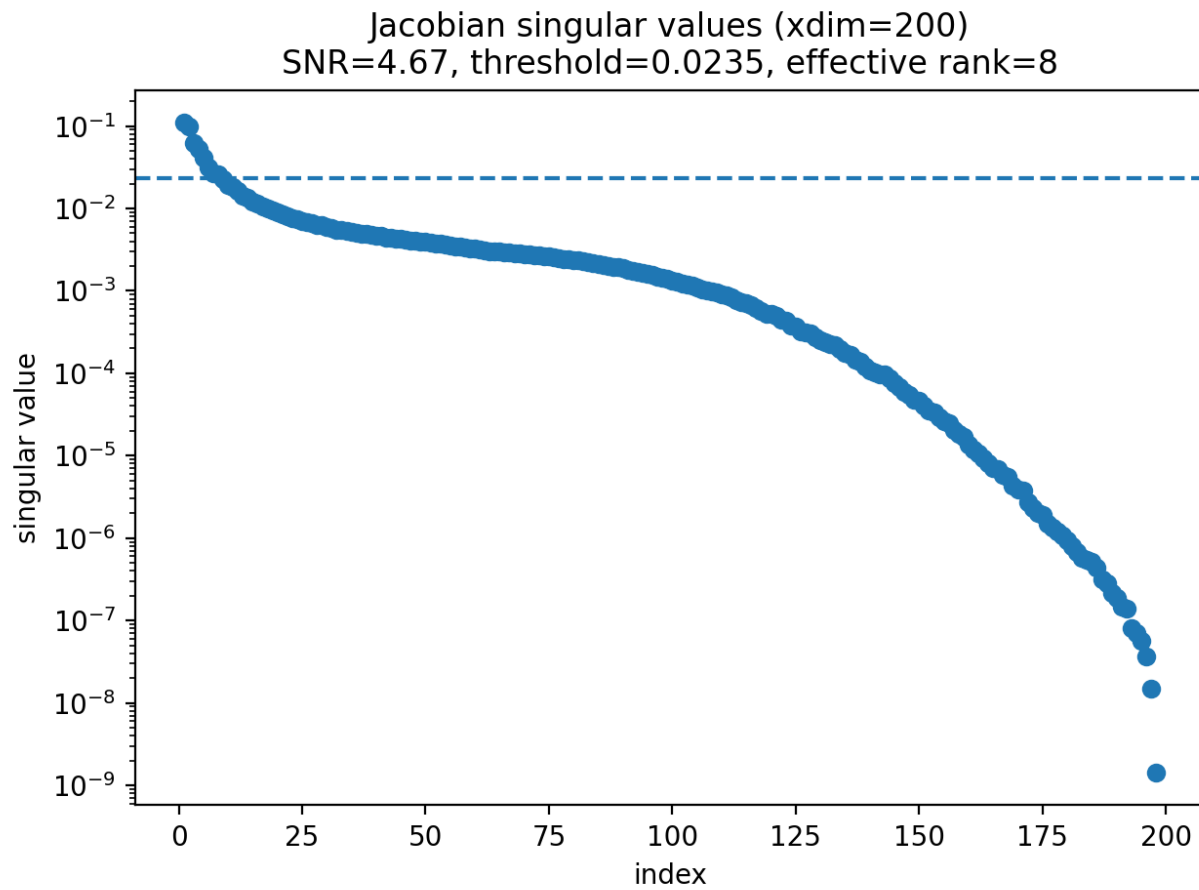
FD scale = 0.05
 noise_std = 0.03 (SNR computed as $\max(|u(T)|)/\text{noise_std}$)

Results:

xdim	nel	ndata	SNR_used	eff_rank	s1 (largest)
20	19	18	4.76	8	1.1793e-01
28	27	26	3.41	6	7.8598e-02
39	38	37	4.85	9	1.0946e-01
54	53	52	5.24	9	1.1613e-01
75	74	73	5	8	1.1188e-01
104	103	102	4.5	7	1.0097e-01
144	143	142	4.54	8	1.0182e-01
200	199	198	4.67	8	1.0973e-01







The effective rank seems to be relatively stable.

Congratulations! Now you are a Bayesian Inverse Problems expert.